

WHC_ModelSqliteKit 详解

WHC_ModelSqliteKit 详解：iOS 本地数据存储的好帮手

一、它是干什么的？

WHC_ModelSqliteKit 是 iOS 开发中一款 **本地数据存储工具库**，核心作用是“将 iOS 模型对象直接映射到 SQLite 数据库”，无需手动编写 SQL 语句，以“面向对象”的方式实现数据的存储与读取。

核心优势（为什么要用它？）

- **简化操作**：避免手动编写复杂 SQL 语句（如建表、插入、查询），通过简单 API 即可完成数据操作；
- **自动映射**：自动将模型属性映射为数据库表字段，无需手动处理数据类型转换；
- **降低门槛**：即使不熟悉 SQLite 语法，也能快速实现本地数据存储，提升开发效率。

二、它怎么用？

使用 WHC_ModelSqliteKit 需遵循“模型配置→数据存储→数据读取”三步流程，以下结合项目实例详解：

1. 第一步：模型类配置（核心）

需让业务模型类实现 **WHC_SqliteInfo 协议**（核心是 `whc_SqliteVersion` 方法），同时可结合 `YYModel` 实现 JSON 字段映射（项目中常用组合）。

项目示例（ZYListDict.m）

```
@implementation ZYListDict
// 1. YYModel 字段映射（可选，用于JSON→模型转换，非WHC_ModelSqliteKit必需）
+ (NSDictionary *)modelCustomPropertyMapper {
```

```

return @{
    @"dictLabel": @"dictLabel", // 模型属性 → JSON字段（名一致，直接映射）
    @"dictValue": @"dictValue"
};
}
// 2. WHC_SqliteInfo 协议必需方法：指定数据库版本号
+ (NSString *)whc_SqliteVersion {
    return @"1.0"; // 版本号为"1.0"，模型结构变更时需更新（如改为"1.1"）
}
@end

```

关键说明

- **whc_SqliteVersion 方法**：用于数据库版本管理，模型属性新增 / 删除 / 修改时，必须更新版本号（如从 @"1.0" 改为 @"1.1"），否则库无法识别表结构变更；
- **YYModel 配合**：项目中常先通过 YYModel 将 JSON 数据转为模型对象，再用 WHC_ModelSqliteKit 存储模型，形成“JSON→模型→数据库”的完整流程。

2. 第二步：数据存储（存数据）

提供单条数据插入、批量数据插入（事务）两种方式，API 简洁，一行代码即可完成。

（1）存储单条数据

```

// 1. 创建模型对象
ZYListDict *model = [ZYListDict new];
model.dictLabel = @"北京市";
model.dictValue = @"110000";
// 2. 插入数据库（返回BOOL值，标识是否成功）
BOOL isSuccess = [WHC_ModelSqlite insert:model];
// 3. 结果判断（可选，建议处理失败场景）
if (isSuccess) {
    NSLog(@"数据存储成功");
} else {
    NSLog(@"数据存储失败");
}

```

(2) 批量存储数据（事务方式，推荐）

批量数据插入时，使用 `inserts:` 方法（基于事务处理，避免多次数据库连接，提升性能）：

```
// 1. 创建模型数组（多条数据）
NSMutableArray *modelArray = [NSMutableArray array];
for (int i = 0; i < 5; i++) {
    ZYListDict *model = [ZYListDict new];
    model.dictLabel = [NSString stringWithFormat:@"城市%@", @(i+1)];
    model.dictValue = [NSString stringWithFormat:@"%06d", 100000 + i];
    [modelArray addObject:model];
}
// 2. 批量插入（事务处理）
BOOL isSuccess = [WHC_ModelSqlite inserts:modelArray];
// 3. 结果判断
if (isSuccess) {
    NSLog(@"批量数据存储成功，共%lu条", modelArray.count);
}
```

3. 第三步：数据读取（取数据）

支持“全量查询、条件查询、排序查询、分页查询”等多种场景，满足不同业务需求。

(1) 查询所有数据

```
// 查询 ZYListDict 表中所有数据（返回模型数组）
NSArray *allModels = [WHC_ModelSqlite query:[ZYListDict class]];
NSLog(@"查询到%lu条数据", allModels.count);
// 遍历使用
for (ZYListDict *model in allModels) {
    NSLog(@"城市名称：%@", 编码：%@", model.dictLabel, model.dictValue);
}
```

(2) 条件查询（带 WHERE 子句）

```
// 查询 “dictLabel = 北京市” 的数据 (SQL WHERE 条件语法)
NSString *whereCondition = @"dictLabel = '北京市'";
NSArray *targetModels = [WHC_ModelSqlite query:[ZYListDict class] where:whereCondition];
if (targetModels.count > 0) {
    ZYListDict *beijingModel = targetModels.firstObject;
    NSLog(@"找到北京市数据，编码：%@", beijingModel.dictValue);
}
```

(3) 排序查询（带 ORDER BY 子句）

```
// 按 “dictValue” 降序排列（从大到小）
NSString *orderClause = @"dictValue DESC";
NSArray *sortedModels = [WHC_ModelSqlite query:[ZYListDict class] order:orderClause];
```

(4) 分页查询（带 LIMIT 子句）

```
// 分页参数：LIMIT "offset,count" → 从第10条开始，取20条（offset从0开始）
NSString *limitClause = @"10,20";
NSArray *pageModels = [WHC_ModelSqlite query:[ZYListDict class] where:@"'" order:@"'" limit:limitClause];
NSLog(@"第2页数据（每页20条），共%lu条", pageModels.count);
```

三、它的核心功能

通过分析库源码，WHC_ModelSqliteKit 核心功能围绕 “自动化、易用性” 设计，关键能力如下：

1. 自动创建数据库表

首次存储某类模型对象时，库会 **通过 Runtime 机制动态获取模型属性**，自动生成建表 SQL，创建对应的 SQLite 表。

核心源码片段（自动建表逻辑）

```

- (void)createTableWithModelClass:(Class)model_class {
    // 1. 获取表名（默认：模型类名 + "_v" + 版本号，如 ZYListDict_v1.0）
    NSString *tableName = [self getTableName:model_class];

    // 2. 解析模型属性，生成字段字典（属性名→属性类型）
    NSDictionary *fieldDict = [self parserModelObjectFieldsWithModelClass:model_class];

    if (fieldDict.count > 0) {
        // 3. 生成建表SQL（默认添加主键 id，自增）
        NSString *mainKey = [self getMainKeyWithClass:model_class]; // 主键字段名（默认"id"）
        NSMutableString *createSql = [NSMutableString stringWithFormat:
            @"CREATE TABLE IF NOT EXISTS %@ (%@ INTEGER NOT NULL PRIMARY KEY
AUTOINCREMENT,",
            tableName, mainKey];

        // 4. 遍历属性，添加字段定义（自动映射数据类型）
        [fieldDict enumerateKeysAndObjectsUsingBlock:^(NSString * _Nonnull propName,
        NSString * _Nonnull propType, BOOL * _Nonnull stop) {
            // 数据类型映射：如 NSString→TEXT，NSNumber→INTEGER
            NSString *sqlType = [self mapPropertyTypeToSqlType:propType];
            [createSql appendFormat:@"%@ %@", propName, sqlType];
        }];

        // 5. 移除最后一个逗号，补全SQL
        [createSql deleteCharactersInRange:NSMakeRange(createSql.length-1, 1)];
        [createSql appendString:@")"];

        // 6. 执行建表SQL
        [self execSql:createSql];
    }
}

```

2. 自动数据类型转换

支持多种数据类型的自动序列化 / 反序列化，无需手动处理，包括：

模型属性类型	SQLite 字段类型	处理逻辑
NSString、NSNumber	TEXT/INTEGER	直接存储字符串 / 数值，无需转换；

NSArray、NSDictionary	BLOB	自动通过 NSKeyedArchiver 序列化为 NSData，存储为 BLOB 类型；读取时自动反序列化；
NSDate	INTEGER	转换为时间戳（毫秒数）存储，读取时自动转为 NSDate；
NSData	BLOB	直接存储为 BLOB 类型；

核心源码片段（复杂类型处理）

```
- (void)bindValue:(id)value toStatement:(sqlite3_stmt *)stmt atIndex:(int)index {
    if (!value) {
        sqlite3_bind_null(stmt, index);
        return;
    }

    // 判断数据类型，执行对应绑定逻辑
    if ([value isKindOfClass:[NSString class]]) {
        sqlite3_bind_text(stmt, index, [value UTF8String], -1, SQLITE_TRANSIENT);
    } else if ([value isKindOfClass:[NSNumber class]]) {
        sqlite3_bind_double(stmt, index, [value doubleValue]);
    } else if ([value isKindOfClass:[NSArray class]] || [value isKindOfClass:[NSDictionary class]]) {
        // 数组/字典：自动序列化
        @try {
            NSData *data = [NSKeyedArchiver archivedDataWithRootObject:value];
            sqlite3_bind_blob(stmt, index, [data bytes], (int)[data length], SQLITE_TRANSIENT);
        } @catch (NSException *exception) {
            // 异常处理：绑定空数据
            sqlite3_bind_blob(stmt, index, [NSData data].bytes, 0, SQLITE_TRANSIENT);
        }
    }

    // ... 其他类型处理（NSDate、NSData等）
}
```

3. 数据库版本管理

通过 `whc_SqliteVersion` 方法实现版本控制，当模型结构变更时，自动更新表结构，保留原有数据（数据迁移）。

核心逻辑

1. 存储模型时，记录 “模型类名 + 版本号”（如 `ZYListDict_v1.0`）；
2. 下次存储前，对比当前版本号与记录的版本号；
3. 若版本号不一致，自动执行 “表结构更新”（如新增字段、删除废弃字段），并更新版本记录；
4. 确保数据不丢失（仅变更字段结构，不删除原有数据）。

4. 事务支持

批量操作（如批量插入、批量删除）时，自动使用事务处理，减少数据库 IO 次数，大幅提升性能。

核心源码片段（事务处理）

```
- (BOOL)inserts:(NSArray *)models {
    if (models.count == 0) return YES;

    // 1. 开启事务
    [self execSql:@"BEGIN TRANSACTION;"];

    BOOL isSuccess = YES;
    // 2. 批量插入（循环调用单条插入逻辑）
    for (id model in models) {
        BOOL success = [self commonInsert:model];
        if (!success) {
            isSuccess = NO;
            break;
        }
    }

    // 3. 事务提交/回滚
    if (isAllSuccess) {
        [self execSql:@"COMMIT;"]; // 全部成功：提交事务
    } else {
        [self execSql:@"ROLLBACK;"]; // 有失败：回滚事务（数据不写入）
    }
}
```

```
}  
  
return isSuccess;  
}
```

四、项目中的实际应用

项目中多个业务模型类均实现了 `whc_SqliteVersion` 方法，表明广泛使用 `WHC_ModelSqliteKit` 进行本地存储，典型应用流程如下：

1. **数据获取**：从网络请求获取 JSON 数据（如 `ZYRentHomeViewController` 的 `cityCnListDict` 方法）；
2. **JSON→模型**：通过 `YYModel` 将 JSON 数据转为业务模型对象（如 `ZYListDict`、`ZYCityArea`）；
3. **模型→数据库**：用 `[WHC_ModelSqlite insert:model]` 或 `inserts:modelArray` 存储模型到本地；
4. **本地优先加载**：下次 App 启动时，先从本地数据库读取数据显示（提升加载速度），再请求网络更新最新数据，最后同步更新本地数据库。

五、工作原理简单解析

`WHC_ModelSqliteKit` 基于 **ORM（对象关系映射）** 思想实现，核心是“模型对象与数据库表的映射关系”，结合 Objective-C Runtime 机制动态处理属性，具体流程如下：

1. **表名映射**：默认表名 = 模型类名 + "_v" + 版本号（如 `ZYListDict_v1.0`）；
2. **字段映射**：模型的每个属性对应表的一个字段，属性名 = 字段名，属性类型通过“自动类型转换”映射为 SQLite 字段类型；
3. **操作映射**：调用库 API（如 `insert:`、`query:`）时，库内部自动生成对应的 SQL 语句（如 `INSERT INTO`、`SELECT * FROM`），执行 SQLite 操作；
4. **结果映射**：查询结果通过 Runtime 机制，自动填充为模型对象的属性，返回模型数组。

六、代码优化建议

结合项目使用场景，提出以下优化建议，提升存储效率与稳定性：

1. 规范版本号管理

- **现状：**项目中所有模型类的 `whc_SqliteVersion` 均返回 `@"1.0"`；
- **建议：**模型属性变更时（新增 / 删除 / 修改类型），必须更新版本号（如从 `@"1.0"` 改为 `@"1.1"`），否则库无法更新表结构，可能导致数据存储异常；
- **示例：**

```
// 模型新增“cityLevel”属性后，版本号从1.0改为1.1
+ (NSString *)whc_SqliteVersion {
    return @"1.1";
}
```

2. 新增字段添加索引（提升查询效率）

- **场景：**对于查询频繁的字段（如 `dictValue` 城市编码），建议添加索引，减少查询耗时；
- **实现方式：**库未直接提供索引 API，可通过 `execSql:` 方法手动执行创建索引 SQL：

```
// 为 ZYListDict 表的“dictValue”字段添加索引
NSString *createIndexSql = @"CREATE INDEX IF NOT EXISTS idx_zylistdict_dictvalue ON
ZYListDict_v1.0 (dictValue)";
[WHC_ModelSqlite execSql:createIndexSql];
```

3. 批量操作强制使用事务

- **场景：**批量插入 / 删除数据（如一次性存储 100 条城市数据）；
- **建议：**必须使用 `inserts:`（批量插入）、`deletes:`（批量删除）方法，避免循环调用 `insert:-/delete:`（非事务方式，多次 IO，性能差）；

4. 完善错误处理

- **现状：**项目中常忽略 `insert:/inserts:` 返回的 BOOL 结果；
- **建议：**检查操作结果，处理失败场景（如日志打印、用户提示），便于问题排查：

```
BOOL isSuccess = [WHC_ModelSqlite inserts:modelArray];
if (!isSuccess) {
    NSLog(@"批量存储失败，模型类：%@", 数据量：%lu", NSStringFromClass([ZYListDict class
]), modelArray.count);
}
```

```
// 可选：显示用户提示
[EasyTextView showErrorText:@"本地数据存储失败，请重试"];
}
```

总结

WHC_ModelSqliteKit 是一款 “低门槛、高效率” 的 iOS 本地存储工具库，通过 “模型配置→简单 API 调用” 即可实现 SQLite 数据操作，大幅简化本地存储开发。项目中结合 YYModel 形成 “JSON→模型→数据库” 的完整数据流转链路，满足日常业务需求。只需规范版本号管理、优化查询效率，即可更好地发挥其优势，提升 App 本地数据存储的稳定性与性能。

（注：文档部分内容可能由 AI 生成）